

```
In [1]: # Step 1: Import necessary libraries
import seaborn as sns
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# Step 2: Load Titanic dataset
titanic = sns.load_dataset('titanic')

# Step 3: Display first few rows of the dataset
print("First few rows of Titanic dataset:")
print(titanic.head())

# Step 4: Check for missing values
print("\nMissing values in the dataset:")
print(titanic.isnull().sum())

# Step 5: Handle missing data
# For numerical columns, we'll impute missing values with the mean.
# For categorical columns, we'll impute missing values with the mode (most frequent)

# Separate numeric and categorical columns
numeric_columns = titanic.select_dtypes(include=['float64', 'int64']).columns
categorical_columns = titanic.select_dtypes(include=['object']).columns

# Impute missing numerical values with the mean
num_imputer = SimpleImputer(strategy='mean')
titanic[numeric_columns] = num_imputer.fit_transform(titanic[numeric_columns])

# Impute missing categorical values with the mode (most frequent)
cat_imputer = SimpleImputer(strategy='most_frequent')
titanic[categorical_columns] = cat_imputer.fit_transform(titanic[categorical_columns])

# Step 6: Check if there are any remaining missing values
print("\nMissing values after imputation:")
print(titanic.isnull().sum())

# Step 7: Feature scaling (Standardize numerical features)
# We will standardize the numeric columns to have a mean of 0 and a standard deviation of 1

scaler = StandardScaler()

# We will scale only the numeric columns
titanic[numeric_columns] = scaler.fit_transform(titanic[numeric_columns])

# Step 8: Display the dataset after scaling
print("\nDataset after scaling numerical features:")
print(titanic.head())
```

First few rows of Titanic dataset:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

Missing values in the dataset:

```

survived      0
pclass        0
sex           0
age          177
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone        0
dtype: int64

```

Missing values after imputation:

```

survived      0
pclass        0
sex           0
age           0
sibsp         0
parch         0
fare          0
embarked      0
class         0
who           0
adult_male    0
deck         688
embark_town   0
alive         0
alone        0
dtype: int64

```

Dataset after scaling numerical features:

	survived	pclass	sex	age	sibsp	parch	fare	\
0	-0.789272	0.827377	male	-0.592481	0.432793	-0.473674	-0.502445	
1	1.266990	-1.566107	female	0.638789	0.432793	-0.473674	0.786845	
2	1.266990	0.827377	female	-0.284663	-0.474545	-0.473674	-0.488854	

```

3  1.266990 -1.566107 female  0.407926  0.432793 -0.473674  0.420730
4 -0.789272  0.827377   male  0.407926 -0.474545 -0.473674 -0.486337

```

```

embarked  class  who  adult_male  deck  embark_town  alive  alone
0         S  Third   man          True   NaN  Southampton   no  False
1         C  First  woman         False   C    Cherbourg   yes  False
2         S  Third  woman         False  NaN  Southampton   yes  True
3         S  First  woman         False   C    Southampton   yes  False
4         S  Third   man          True  NaN  Southampton   no   True

```

```

In [11]: import pandas as pd
from sklearn.preprocessing import LabelEncoder
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Assuming 'X' and 'y' are your features and target respectively

# Check column names and categorical columns
print("Column names in X:", X.columns)
print("Categorical columns:", X.select_dtypes(include=['object']).columns)

# Handle missing values if necessary
# For example, you can fill missing values with a placeholder like 'Unknown' for ca
if X['deck'].dtype == 'category':
    X['deck'] = X['deck'].cat.add_categories('Unknown') # Add 'Unknown' as a categ
else:
    X['deck'] = X['deck'].astype('category') # Convert to category type if not alr
    X['deck'] = X['deck'].cat.add_categories('Unknown') # Add 'Unknown' as a categ

X['deck'].fillna('Unknown', inplace=True) # Fill missing values with 'Unknown'

# Apply LabelEncoder for ordinal categorical features (if needed)
encoder = LabelEncoder()
for column in ['class', 'who']: # Replace with actual ordinal columns if needed
    X[column] = encoder.fit_transform(X[column])

# Apply one-hot encoding for nominal categorical features (Like 'deck', 'embark_tow
X = pd.get_dummies(X, drop_first=True)

# Check the data after encoding
print("Data after encoding:\n", X.head())

# Apply SMOTE for oversampling
smote = SMOTE(sampling_strategy='auto', random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Check the shape of the resampled data
print("Resampled X shape:", X_resampled.shape)
print("Resampled y shape:", y_resampled.shape)

# Split the resampled data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_

# Example classifier (e.g., RandomForestClassifier)
from sklearn.ensemble import RandomForestClassifier

```

```

classifier = RandomForestClassifier(random_state=42)

# Train the classifier
classifier.fit(X_train, y_train)

# Predict on the test data
y_pred = classifier.predict(X_test)

# Evaluate the performance
print("Classification Report:\n", classification_report(y_test, y_pred))

```

```

Column names in X: Index(['pclass', 'age', 'sibsp', 'parch', 'fare', 'class', 'who',
'adult_male',
      'deck', 'embark_town', 'alive', 'alone', 'sex_male', 'embarked_Q',
      'embarked_S'],
      dtype='object')

```

```

Categorical columns: Index([], dtype='object')

```

Data after encoding:

	pclass	age	sibsp	parch	fare	class	who	adult_male	\
0	0.827377	-0.592481	0.432793	-0.473674	-0.502445	2	1	True	
1	-1.566107	0.638789	0.432793	-0.473674	0.786845	0	2	False	
2	0.827377	-0.284663	-0.474545	-0.473674	-0.488854	2	2	False	
3	-1.566107	0.407926	0.432793	-0.473674	0.420730	0	2	False	
4	0.827377	0.407926	-0.474545	-0.473674	-0.486337	2	1	True	

	embark_town	alive	...	sex_male	embarked_Q	embarked_S	deck_B	deck_C	\
0	2	0	...	True	False	True	False	False	
1	0	1	...	False	False	False	False	True	
2	2	1	...	False	False	True	False	False	
3	2	1	...	False	False	True	False	True	
4	2	0	...	True	False	True	False	False	

	deck_D	deck_E	deck_F	deck_G	deck_Unknown
0	False	False	False	False	True
1	False	False	False	False	False
2	False	False	False	False	True
3	False	False	False	False	False
4	False	False	False	False	True

[5 rows x 21 columns]

C:\Users\nagen\AppData\Local\Temp\ipykernel_35296\597404513.py:21: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```

X['deck'].fillna('Unknown', inplace=True) # Fill missing values with 'Unknown'

```

```
Resampled X shape: (1098, 21)
Resampled y shape: (1098,)
Classification Report:
              precision    recall  f1-score   support

    0           1.00        1.00        1.00        103
    1           1.00        1.00        1.00        117

 accuracy              1.00        220
 macro avg           1.00        1.00        1.00        220
weighted avg           1.00        1.00        1.00        220
```

In []: